Design and Development of an Immersive Virtual Reality Eight-Ball Pool Simulation

A Physics-Driven Billiards Environment Implemented in Unity 3D Using the XR Interaction Toolkit

Mayank Agrawal
Roll No: 230639
Indian Institute of Technology Kanpur
mayankag23@iitk.ac.in

***Abstract*—Traditional video game simulations of billiards often suffer from a fundamental limitation: players interact through a flat display rather than within the physical space of the table. This limitation is particularly evident in cue sports, where spatial judgment, angle estimation, and force control are critical. This paper presents the design and development of a Virtual Reality (VR)–based eight-ball pool simulation implemented in Unity 3D using C# and the XR Interaction Toolkit. The system incorporates Rigidbody-based physics for realistic ball dynamics, a drag-based cue interaction model for shot power control, procedural ball initialization, and texture-based ball identification. An adaptive camera system assists shot alignment, while a lightweight user interface tracks player turns and scores. The application supports both desktop (mouse and keyboard) and immersive VR interaction modes. Rather than aiming for high-precision competitive simulation, this work focuses on demonstrating how VR-based interaction design can enhance user presence and engagement in precision-based sports environments.**

***Index Terms*—Virtual Reality (VR), Eight-Ball Pool, Billiards Simulation, Unity 3D, XR Interaction Toolkit, Physics-Based Simulation, Immersive Interaction, Game Development**

I. Introduction

Cue sports such as billiards are inherently spatial activities that rely on precise angle estimation, force control, and physical positioning around the table. However, conventional digital implementations of pool games are constrained by two-dimensional displays, where interaction is mediated through fixed camera views and simplified input mechanisms. As a result, these systems often reduce gameplay to indirect drag-and-release interactions, limiting the player's ability to engage with the embodied aspects of the sport.

Virtual Reality (VR) provides an opportunity to address this limitation by enabling direct, physically grounded interaction within a simulated three-dimensional environment. By allowing users to position themselves around a virtual table, align shots through natural head and body movement, and manipulate a cue using motion-tracked controllers, VR systems can more closely approximate the perceptual and motor skills involved in real-world billiards.

This work presents the design and development of a VR-based eight-ball pool simulation created using Unity 3D. The system supports both desktop (mouse and keyboard) and immersive VR interaction modes, enabling comparative exploration of interaction fidelity across platforms. The implementation incorporates physics-based ball dynamics using Unity's Rigidbody system, a drag-based cue interaction model adaptable to multiple input modalities, procedural ball initialization, and a user interface for real-time game state feedback.

The primary objective of this project is not to achieve high-precision competitive simulation, but to investigate how interaction design in VR can enhance spatial engagement and user experience in precision-based sports simulations. This paper details the system architecture, interaction design decisions, and technical challenges encountered during development.

II. Literature Review

A. VR in Gaming and Simulation

The adoption of Virtual Reality (VR) in gaming and simulation has expanded significantly with the increased accessibility of consumer-grade head-mounted displays. Early research primarily focused on the concepts of presence and immersion, particularly in first-person virtual environments [1]. These studies established that VR can enhance user engagement by creating a strong sense of spatial presence.

More recent research has shifted toward interaction fidelity, especially in tasks requiring fine motor control. Studies have investigated how users perform precision-based actions using handheld controllers compared to real-world physical tools [2]. Findings indicate that interaction accuracy and user satisfaction improve when virtual tools closely replicate real-world behavior and affordances.

Within this context, sports simulation in VR has emerged as a growing area of research. Applications include systems for training and rehabilitation, such as golf swing analysis tools and cricket batting simulators. A key challenge across these systems is the discrepancy between controller-based input and the physical characteristics of real-world sporting equipment. Addressing this gap through improved physics modeling, visual feedback, and interaction design remains an active area of investigation [**?**].

This work builds upon these findings by exploring interaction fidelity in a cue-based sport, specifically focusing on how VR can enhance spatial reasoning and motor engagement in an eight-ball pool simulation.

B. Physics Engines in Simulation

Unity's PhysX-based Rigidbody system is widely used in interactive simulation due to its real-time performance and acceptable level of physical fidelity [3]. In the context of billiards simulation, both academic and commercial systems often employ custom physics models to accurately represent rolling friction, elastic collisions, and spin effects such as topspin, backspin, and sidespin. High-fidelity simulators, including Virtual Pool 4 and ShootersPool, achieve near-physical realism by implementing specialized collision solvers and physics models on top of standard game engines.

In contrast, the present work adopts Unity's built-in Rigidbody physics with carefully tuned material parameters, including friction and restitution, to achieve visually plausible ball dynamics. This design choice reflects a deliberate trade-off: rather than targeting sub-millimeter physical accuracy, the system prioritizes real-time performance and interactive immersion. The focus is therefore on creating a responsive and engaging user experience rather than a professional training simulator.

C. XR Interaction Toolkit and Unity VR Development

Unity's XR Interaction Toolkit (XRI) provides a high-level abstraction layer for developing VR applications across multiple hardware platforms, including Meta Quest, Valve Index, and PC-based VR systems [4]. The toolkit includes components for locomotion, ray-based interaction, direct manipulation, and object grabbing, enabling rapid prototyping of interactive environments.

In this project, XRI is utilized primarily for locomotion and controller input handling, while the core cue interaction mechanism is implemented using custom scripts based on controller transform data. This hybrid approach is necessary because standard XRI interaction models are optimized for discrete object manipulation, whereas billiards requires continuous, force-sensitive interaction. By extending XRI with custom interaction logic, the system supports more natural cue control aligned with the physical expectations of the sport.

III. Methodology

A. Development Approach

The project followed an iterative development process based on incremental prototyping and testing. The initial prototype consisted of a desktop-based pool simulation with basic ball physics and mouse-controlled cue interaction. Subsequent features, including VR interaction, texture mapping, shot power control, user interface elements, and locomotion, were integrated progressively. Each component was tested independently prior to integration into the main system to ensure stability and modularity.

This iterative approach led to design decisions driven by empirical testing rather than predefined specifications. For example, the cue interaction system was divided into separate implementations for desktop and VR input after attempts to unify both interaction modes resulted in increased complexity and reduced maintainability. This separation enabled cleaner abstraction of input handling across platforms.

B. Key Debugging Challenges

Several technical challenges were encountered during development. One major issue involved object scaling and its interaction with Unity's physics engine. At real-world scale (approximately 57 mm ball diameter), collision behavior was unstable, with occasional clipping during high-speed interactions. This was resolved by scaling the scene to approximately ten times real-world dimensions and recalibrating physical parameters accordingly.

A second challenge involved the interaction between VR locomotion and cue control. Early implementations resulted in unintended cue drift when the player moved within the XR environment, due to the cue being parented to a controller transform within a moving reference frame. This issue was addressed by decoupling the cue representation from the controller hierarchy and explicitly computing its world-space position each frame.

A third issue involved visual artifacts on ball surfaces caused by texture seams. This was traced to inconsistencies in the UV mapping of imported mesh assets. Re-importing the meshes with automatic UV generation resolved the issue, with a minor trade-off in texture sharpness.

C. Design Decisions

The system supports both desktop and VR interaction modes, enabling flexible testing and comparison of interaction fidelity. While this introduces additional implementation complexity, it allows consistent development without continuous access to VR hardware and provides a baseline for evaluating interaction differences.

The cue interaction system employs a drag-based input model in which shot power is proportional to the displacement of the cue from the ball. This displacement is mapped linearly to an impulse force applied to the cue ball using Unity's physics system. The use of an impulse-based force model ensures an instantaneous transfer of energy, closely approximating the behavior of a real cue strike.

Ball dynamics are simulated using Unity's Rigidbody component with carefully tuned parameters. Each ball uses a mass of 1 unit, with linear and angular damping values set to 0.6 to balance motion persistence and gradual deceleration. Continuous dynamic collision detection is enabled to reduce tunneling effects during high-speed impacts, and interpolation is used to improve visual smoothness.

Surface interactions are controlled through physics materials assigned to both the balls and the table. The ball material uses low friction values (0.1) and moderate restitution (0.15) to allow realistic rolling and bouncing behavior, while the table surface uses slightly higher friction (0.2) and zero restitution to simulate energy loss during contact. These parameters were empirically tuned to achieve visually plausible and stable interactions within the constraints of real-time simulation.

Overall, the system prioritizes interaction fidelity and responsiveness over strict physical accuracy, with design choices aimed at enhancing user immersion and playability in both desktop and VR environments.

IV. System Architecture and Implementation

A. Ball Physics System

Each of the sixteen balls in the simulation is implemented as a Unity GameObject with a `Rigidbody` component and a `SphereCollider`. The physics system is based on Unity's PhysX engine, with parameters tuned to achieve stable and visually plausible billiards behavior in real time.

The Rigidbody configuration uses a mass of 1 unit, with linear and angular damping values set to 0.6 to balance motion persistence and gradual deceleration. Continuous dynamic collision detection is enabled to prevent tunneling during high-speed interactions, and interpolation is used to improve rendering smoothness.

Surface interactions are controlled using physics materials assigned to both the balls and the table. The ball material is configured with low friction values (dynamic and static friction of 0.1) and moderate restitution (bounciness of 0.15), allowing smooth rolling and controlled rebound. The table surface uses slightly higher friction values (0.2) and zero restitution to simulate energy loss during contact with the playing surface.

TABLE I
Physics Parameters Used in the Simulation

Parameter	Ball	Table Surface
Mass	1	–
Dynamic Friction	0.1	0.2
Static Friction	0.1	0.2
Bounciness	0.15	0.0
Linear Damping	0.6	–
Angular Damping	0.6	–
Collision Mode	Continuous Dynamic	–

The cue ball is distinguished from the numbered balls using a dedicated identifier within the ball script. Pocket detection is implemented using trigger colliders, and when the cue ball is pocketed, it is repositioned to a predefined starting location with its velocity reset.

Unlike high-fidelity billiards simulators, the system does not implement custom rolling friction models or spin dynamics. Instead, damping parameters and physics material properties are empirically tuned to approximate realistic motion while maintaining computational efficiency.

B. Texture Mapping System

Ball identification is achieved through texture-based differentiation of numbered billiard balls. A set of sixteen textures (balls 1–15 and the cue ball) is applied using a `BallTextureManager` script. The script maintains an indexed array of `Material` references, which are assigned to each ball GameObject during initialization.

The distinction between solid balls (1–7) and striped balls (9–15) is encoded directly within the texture assets rather than being generated procedurally. The 8-ball uses a distinct black texture, while the cue ball is rendered using a white material with slight specular properties to reflect scene lighting.

C. Game Setup System

At the start of each game, the fifteen numbered balls are arranged in a standard triangular rack formation. The rack configuration follows a 5–4–3–2–1 layout, with positions computed procedurally in the `GameSetup` script based on the foot spot location of the table.

Ball positions are determined using offsets derived from the ball diameter in scene space, ensuring consistent spacing and preventing initial overlap between colliders. The procedural approach allows the rack to be generated dynamically at runtime, avoiding the need for manually placed configurations.

To introduce variability, a small random perturbation is applied to each ball's position within a limited range (approximately 2% of the ball diameter). This prevents identical break patterns across repeated games and improves the perceived realism of the simulation without significantly affecting stability.

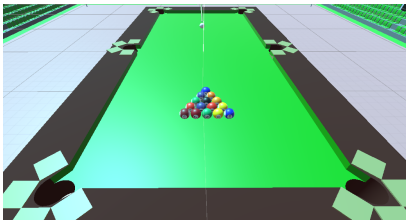

Fig. 1. Procedural rack formation at game start showing a triangular arrangement of fifteen balls.

D. Camera and Cue Control System

The camera system is implemented differently for desktop and VR interaction modes. In desktop mode, a custom `CameraController` script enables orbital camera movement around the cue ball. The camera maintains a fixed radial distance from the ball while allowing rotational control via mouse input. Vertical adjustment is supported through scroll input, enabling the user to modify viewing angle for shot alignment.

In VR mode, the camera is directly coupled to the XR Origin's head-tracking transform, providing a first-person perspective driven by the user's physical head movement. Unlike desktop mode, no artificial camera orbit is applied. This approach allows the player to align shots through natural body positioning and head movement, resulting in a more intuitive and immersive interaction.

The cue stick is represented as a cylindrical mesh controlled by a dedicated cue controller system. In desktop mode, the cue orientation is computed using a raycast from the camera through the mouse cursor onto a horizontal plane, defining a target point in world space. The cue is then aligned along the vector from this point to the cue ball, enabling directional aiming.

In VR mode, the cue's position and orientation are directly driven by the right-hand controller transform. This enables continuous, physically grounded manipulation of the cue, closely matching real-world interaction patterns.

Shot power in both modes is implemented using a drag-based interaction model. The system computes the displacement between the cue tip and the center of the cue ball, and maps this distance linearly to an impulse force applied to the ball. The displacement is clamped to a predefined maximum to ensure stability and prevent excessive velocities. This unified power model ensures consistent behavior across both desktop and VR interaction modes.

E. VR Interaction System

The VR interaction layer is implemented using Unity's XR Interaction Toolkit (XRI, version 2.5). An `XR Origin` GameObject serves as the root of the camera and controller hierarchy, enabling tracked head and hand movement within the virtual environment. Locomotion is achieved using a `Continuous Move Provider` mapped to the left controller thumbstick, while rotational control is implemented via a snap-turn provider on the right thumbstick.

The player is initialized at one end of the pool table at an approximate table-height position. The XR Origin vertical offset is set to 1.65 meters to approximate average user eye height, allowing immediate interaction without requiring explicit floor calibration.

Controller input for cue interaction is handled using the `InputActionReference` system provided by XRI. The right controller is used for primary interaction, with the trigger input initiating the drag-based power mechanism and a button input used to confirm shot execution. The cue representation provides continuous visual feedback for shot power, as its displacement from the cue ball increases proportionally with the applied input.

This interaction model enables continuous, physically grounded manipulation of the cue, aligning closely with real-world motor behavior and improving user immersion compared to discrete input-based systems.

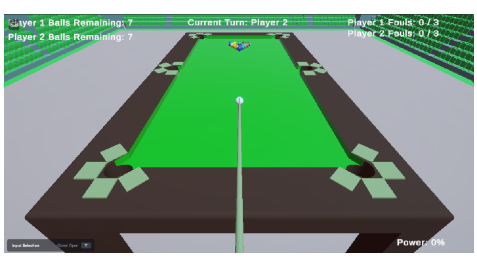

Fig. 2. First-person VR view of the pool table, showing cue alignment and interaction from the player's perspective.

F. UI System

The user interface is implemented using a World Space canvas positioned above the pool table, providing real-time feedback on game state. The UI displays the current player's turn, shot power, and ball count for each player. Shot power is represented as a continuous fill-based indicator, updated dynamically based on the cue displacement.

The use of a World Space canvas enables consistent visibility in both desktop and VR modes without relying on screen-space overlays. In VR mode, the UI is anchored to a fixed position near the table rather than being attached to the user's viewpoint, allowing natural head movement and reducing visual clutter.

To improve usability, a gaze-based highlighting mechanism is implemented. A ray cast from the XR camera origin detects when the user is looking toward the interface, triggering a visual emphasis effect. This provides subtle feedback without interrupting gameplay and supports intuitive interaction within the immersive environment.

TABLE II
SYSTEM TECHNICAL SPECIFICATIONS

Specification	Details
Game Engine	Unity 6 (6000.x)
Programming Language	C#
VR SDK	XR Interaction Toolkit v2.5
Supported Platforms	Windows, PC VR (via Link)
Physics Engine	Unity PhysX (Rigidbody-based)
Ball Count	16 (15 numbered + cue ball)
Input Modes	Mouse/Keyboard, VR Controllers
UI Mode	World Space Canvas
Render Pipeline	Universal Render Pipeline (URP)

The interface also reflects rule-based game states, including player turns, foul counts, and win/loss messages, ensuring continuous feedback during gameplay.

G. Game Logic and Rule System

The system implements a simplified rule framework for two-player eight-ball pool, managed through a centralized game manager script. The logic includes turn-based gameplay, foul detection, win/loss conditions, and state-driven UI updates.

Each player is assigned a set of balls (Player 1: 1–7, Player 2: 9–15), with the remaining ball count tracked dynamically. A foul system is implemented with a configurable limit (three fouls), after which the player automatically loses the game.

Several gameplay events are handled explicitly. Pocketing the cue ball results in a foul and triggers a positional reset of the cue ball to a predefined starting location. Pocketing the 8-ball before clearing all assigned balls results in an immediate loss, while pocketing it after clearing all balls results in a win.

The system also supports camera state transitions during gameplay. A cue-aligned camera is used during aiming, while an overhead camera is activated after shot execution to observe ball motion. A restart mechanism is provided to reset the game state after completion.

These rule-based systems ensure structured gameplay while maintaining simplicity suitable for an interactive VR experience.

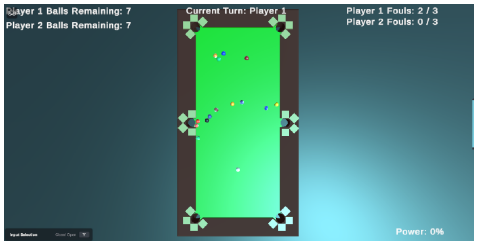

Fig. 3. Gameplay interface showing player turns, remaining balls, foul counts, and shot alignment.

V. Results and Analysis

A. Functional Verification

Both desktop and VR modes were evaluated through approximately thirty play sessions conducted during development. The desktop mode reached stability early and was used as a baseline for tuning physics parameters. Ball motion and collision behavior were visually plausible for standard interactions, including straight shots, cushion rebounds, and combination plays. However, due to the absence of explicit spin modeling, advanced shot types such as massé and swerve shots were not supported.

The VR mode required additional iteration to achieve comparable stability. Issues related to object scaling, locomotion-induced drift, and texture artifacts—identified during development—were resolved prior to the final build. In the final system, the VR experience remained stable across extended sessions. No noticeable motion discomfort was observed during testing under standard locomotion settings.

B. Interaction Quality

The drag-based cue interaction model demonstrated strong performance in VR. Informal user testing with peers indicated that the interaction felt intuitive and closely resembled real-world cue handling. The use of displacement-based power control, without reliance on explicit numerical indicators, encouraged skill-based interaction and improved user engagement.

In contrast, the same interaction model was less effective in desktop mode. The limited range of mouse movement resulted in rapid power scaling, reducing precision and control. Alternative interaction techniques, such as time-based power accumulation, were considered but not implemented due to project constraints.

C. Known Limitations

Despite achieving functional stability, several limitations remain. The physics system does not model angular momentum or spin, limiting the realism of advanced shot techniques and position control. Pocket detection is implemented using trigger volumes, which may fail to register edge-case interactions at low velocities or steep entry angles.

The multiplayer system is restricted to a local hot-seat configuration and does not support networked or multi-user VR interaction. Additionally, the table and cushion geometry are modeled as static rigid bodies without deformation or advanced rebound modeling, which can result in non-ideal collision responses at high velocities.

These limitations highlight areas for future work, particularly in improving physical realism, interaction precision, and multi-user capabilities.

VI. Discussion

A. VR Realism vs. Performance Trade-offs

A key design consideration throughout development was the trade-off between physical realism and real-time performance. High-fidelity billiards simulation requires modeling factors such as spin, cloth friction, cushion elasticity, and energy dissipation, often through custom physics solvers or numerical integration methods beyond the capabilities of Unity's default PhysX system [6].

In this work, the decision to exclude spin dynamics was made deliberately in order to prioritize system stability, responsiveness, and ease of interaction. While this limits the accuracy of advanced shot behavior, it enables a smoother and more accessible gameplay experience. Given the project's focus on interaction design in VR, this trade-off is appropriate. However, applications targeting professional training or competitive simulation would require more advanced physical modeling.

B. User Experience Observations

Informal user testing revealed that players quickly adapted to the absence of spin by adjusting their aiming strategies. This suggests that even a simplified physics model, when combined with immersive interaction, can effectively engage spatial reasoning and motor coordination.

In contrast to traditional desktop-based billiards games, users in VR demonstrated embodied interaction behaviors, including repositioning themselves around the table, aligning their viewpoint physically, and adjusting stance for shot accuracy. These behaviors indicate a higher level of spatial engagement and reinforce the benefits of VR-based interaction for precision tasks.

The World Space UI design proved effective overall, though its initial placement interfered with shot alignment. Relocating the interface to a fixed position at the far end of the table, slightly elevated from the surface, improved visibility while minimizing distraction during gameplay.

C. Scope and Generalization

The system architecture developed in this project—combining physics-material-tuned Rigidbodies, displacement-based force input, XR Interaction Toolkit locomotion, and World Space UI—demonstrates a level of generality applicable to other table-based sports simulations.

For example, adapting the system to snooker would primarily involve scaling the table geometry and modifying ball configurations, while an air hockey implementation would require adjustments to friction parameters and collision dynamics. Despite these domain-specific modifications, the underlying

interaction and system design framework remains transferable with minimal changes.

VII. Conclusion

This paper presented the design and implementation of a Virtual Reality (VR)–based eight-ball pool simulation developed using Unity 3D, C#, and the XR Interaction Toolkit. The system supports both desktop and immersive VR interaction modes and incorporates physics-based ball dynamics, a displacement-driven cue control mechanism, procedural rack initialization, texture-based ball identification, and a World Space user interface.

The development process followed an iterative, problem-driven approach, addressing several technical challenges related to physics scaling, interaction consistency, and rendering artifacts. The resulting system achieves functional stability and provides a responsive and engaging user experience. Informal evaluation indicates that the VR mode offers a significantly higher degree of immersion and spatial interaction compared to its desktop counterpart.

Future work can extend the system in several directions. Incorporating a physically accurate spin model based on established billiards dynamics would enhance realism and enable advanced shot techniques. Supporting networked multiplayer using frameworks such as Unity Netcode for GameObjects would allow shared VR gameplay. Additionally, integrating AI-based opponents and haptic feedback mechanisms could further improve usability and player engagement.

Overall, this work demonstrates that VR-based interaction design can enhance user experience in precision-based sports simulations, even in the presence of simplified physical modeling. The findings suggest that embodied interaction—through natural head movement, controller-based input, and spatial positioning—plays a critical role in improving engagement and perceived realism beyond what is achievable in traditional screen-based systems.

Acknowledgment

The author sincerely thanks Prof. Amar Kumar Behera, instructor of the *DES 643* course at the Department of Design, IIT Kanpur, for his valuable guidance and feedback throughout the development process. The author also acknowledges the contributions of classmates who participated in informal playtesting sessions.

The stadium environment model was sourced from CGTrader (“Dunwell Woodfield Stadium”), while the pool table and ball models were created by the author using Blender.

Source Code

The project source code is available at: https://github.com/mayankag23/VR-Eightball-Pool-Simulation-Game

References

[1] M. Slater and M. Usoh, “Taking steps: The influence of a walking technique on presence in virtual reality,” *ACM Trans. Comput.-Hum. Interact.*, vol. 2, no. 3, pp. 201–219, 1995.

[2] J. Jerald, *The VR Book: Human-Centered Design for Virtual Reality*. Morgan & Claypool Publishers, 2015.

[3] Unity Technologies, “Unity Manual: Rigidbody physics overview,” [Online]. Available: https://docs.unity3d.com/Manual/PhysicsSection.html. Accessed: 2026.

[4] Unity Technologies, “XR Interaction Toolkit documentation,” [Online]. Available: https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@2.5/manual/index.html. Accessed: 2026.

[5] Unity Technologies, “Universal Render Pipeline overview,” [Online]. Available: https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@14.0/manual/index.html. Accessed: 2026.

[6] S. Mathaven, M. S. Hasan, and R. E. Bhansali, “A dynamical model of billiards with spin and friction effects,” *Proc. Inst. Mech. Eng. C*, vol. 224, no. 9, pp. 1923–1938, 2009.

[7] Poly Haven, “Free HDRI, textures, and 3D assets,” [Online]. Available: https://polyhaven.com/. Accessed: 2026.

[8] CGTrader, “Dunwell Woodfield Stadium 3D Model,” [Online]. Available: https://www.cgtrader.com/free-3d-models/exterior/stadium/dunwell-woodfield-stadium--2. Accessed: 2026.

[9] Blender Foundation, “Blender – Open-source 3D creation suite,” [Online]. Available: https://www.blender.org/. Accessed: 2026.

[10] Git Documentation, “Connect a Unity project to GitHub Desktop,” [Online]. Available: https://learn.unity.com/course/collaborate-with-github-desktop/tutorial/connect-a-unity-project-to-github-desktop. Accessed: 2026.

[11] English Pool Association, “World Eightball Pool Rules,” [Online]. Available: https://www.epa.org.uk/images/rules/Pool%20Rules%20B3%20poster%20Oct%2019.pdf. Accessed: 2026.

[12] DES 643 Course Notes, “Design for Augmented Reality and Virtual Reality,” Department of Design, Indian Institute of Technology Kanpur, 2025.